

Elemental Quest

UML Class Diagram Design

CMPT 276 - Group 12

February 15, 2026

Complete System Overview

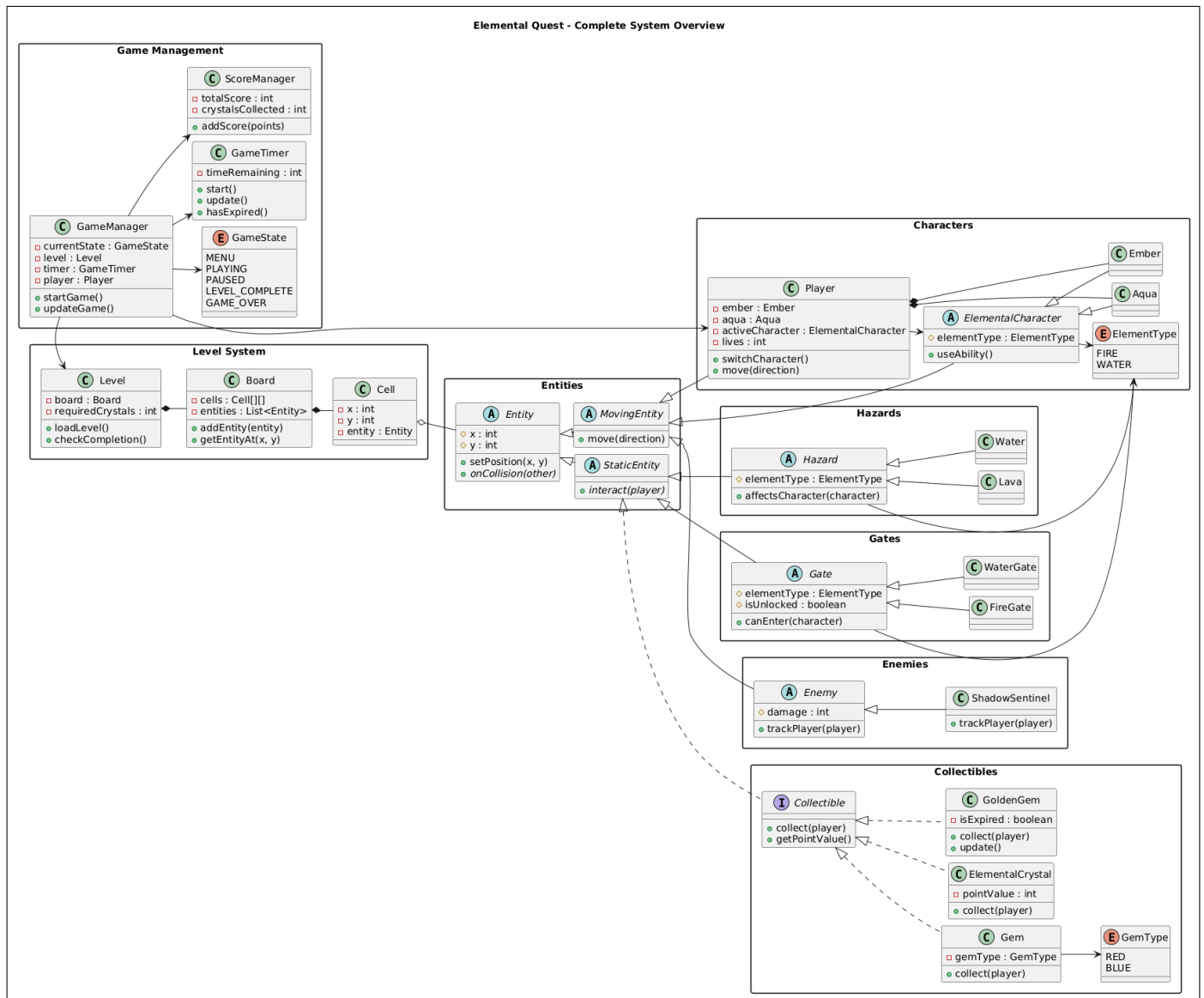


Figure 1: Complete system overview showing all packages and relationships

The complete system architecture organizes Elemental Quest into eight distinct packages: Game Management, Level System, Entities, Characters, Enemies, Hazards, Collectibles, and Gates. This modular organization enables independent development and testing of subsystems while maintaining clear integration points through well-defined interfaces and relationships. The design emphasizes separation of concerns, with each pack-

age containing classes that share related responsibilities and collaborate within bounded contexts. All game entities inherit from a common Entity base class, which branches into MovingEntity and StaticEntity to optimize different behavioral patterns. This high-level structure provides a roadmap for the implementation, showing how game flow (management), spatial organization (level system), interactive objects (entities), and gameplay mechanics (characters, enemies, hazards, collectibles, gates) work together to create the complete game experience.

Game Management & Core Systems

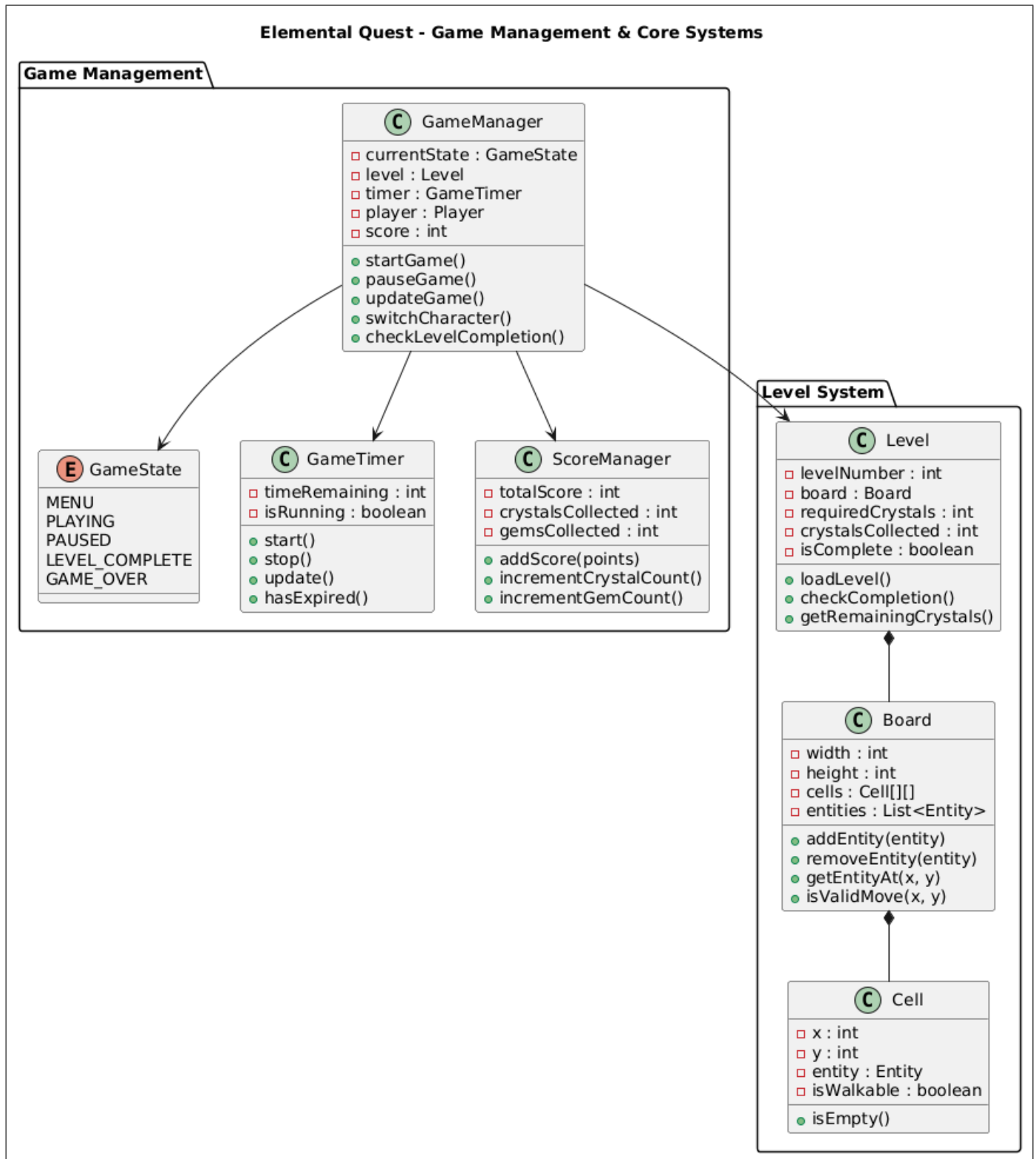


Figure 2: Game Management and Level System showing coordination and state management

The Game Management package provides centralized control over game flow through the GameManager class, which serves as a facade for coordinating all major game systems. The GameManager maintains references to the Level, GameTimer, Player, and ScoreManager, allowing it to orchestrate game initialization, updates, and state transitions through a clean interface. The GameState enumeration implements the State pattern, preventing invalid state combinations and making transitions explicit (MENU to PLAYING to LEVEL_COMPLETE or GAME_OVER). Separating ScoreManager from GameManager follows the Single Responsibility Principle, isolating scoring logic and collection tracking (crystals and gems) from game flow control. The GameTimer exists as an independent component that can be paused, reset, or queried without coupling to other systems, supporting features like time-based bonuses or level time limits. The Level System package models the spatial game environment, with the Level class acting as a facade for board state and completion validation. The Board maintains a 2D Cell array for efficient position-based queries and an Entity list for iteration during updates. Each Cell encapsulates its position, occupying entity, and walkability, simplifying collision detection and movement validation. This architecture enables the Level to track progress (crystalsCollected versus requiredCrystals) and validate completion conditions independently of the GameManager's broader responsibilities.

Character & Entity System

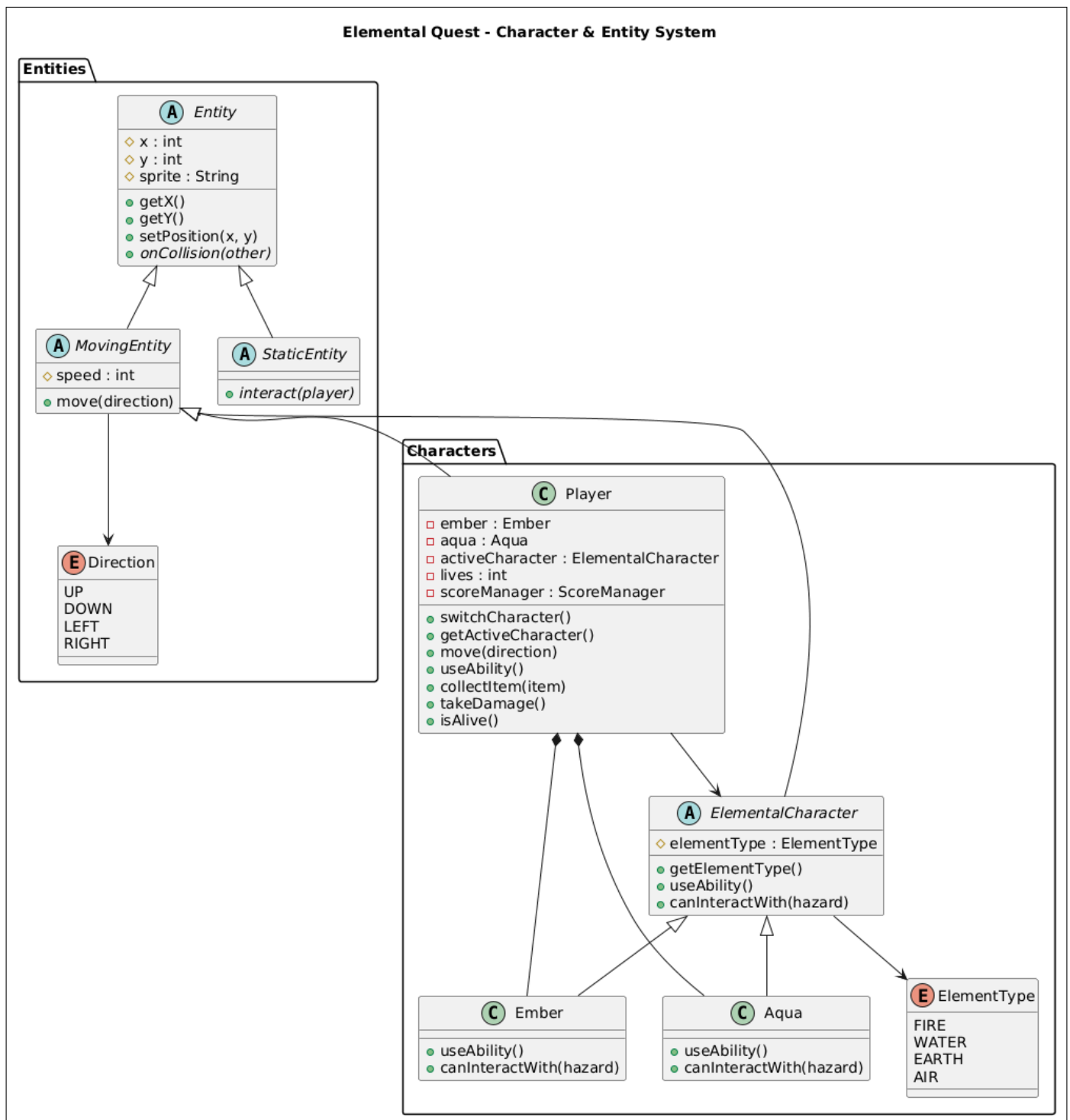


Figure 3: Character system showing Player composition with Ember and Aqua

The character system implements composition over inheritance as its core design principle, with the Player class containing both Ember and Aqua rather than being an ElementalCharacter itself. This critical design decision addresses the gameplay requirement that both characters exist simultaneously on the board at different positions with independent health states and must both reach their respective gates to complete a level. The Player class represents the human controller who switches between commanding Ember and Aqua by updating the activeCharacter reference, while all movement and ability commands are delegated to whichever character is currently active. Each ElementalCharacter subclass maintains its own position, element type, and state, inheriting movement capabilities from MovingEntity while specializing behavior through the useAbility and canInteractWith methods. The ElementType enumeration provides type-safe element identification (FIRE, WATER, EARTH, AIR) that extends naturally to future character additions without modifying existing code. The abstract Entity base class ensures all game objects share common positional and collision interfaces, while the MovingEntity extension adds speed and directional movement for objects that change position. If Player were an ElementalCharacter instead of containing them, managing two simultaneous characters would require maintaining two Player instances (confusing semantics) or destroying and recreating characters on each switch (losing state), both of which violate clean object-oriented design. The Direction enumeration standardizes movement input, allowing consistent handling of UP, DOWN, LEFT, RIGHT commands across all moving entities in the system.

Collectibles & Gates

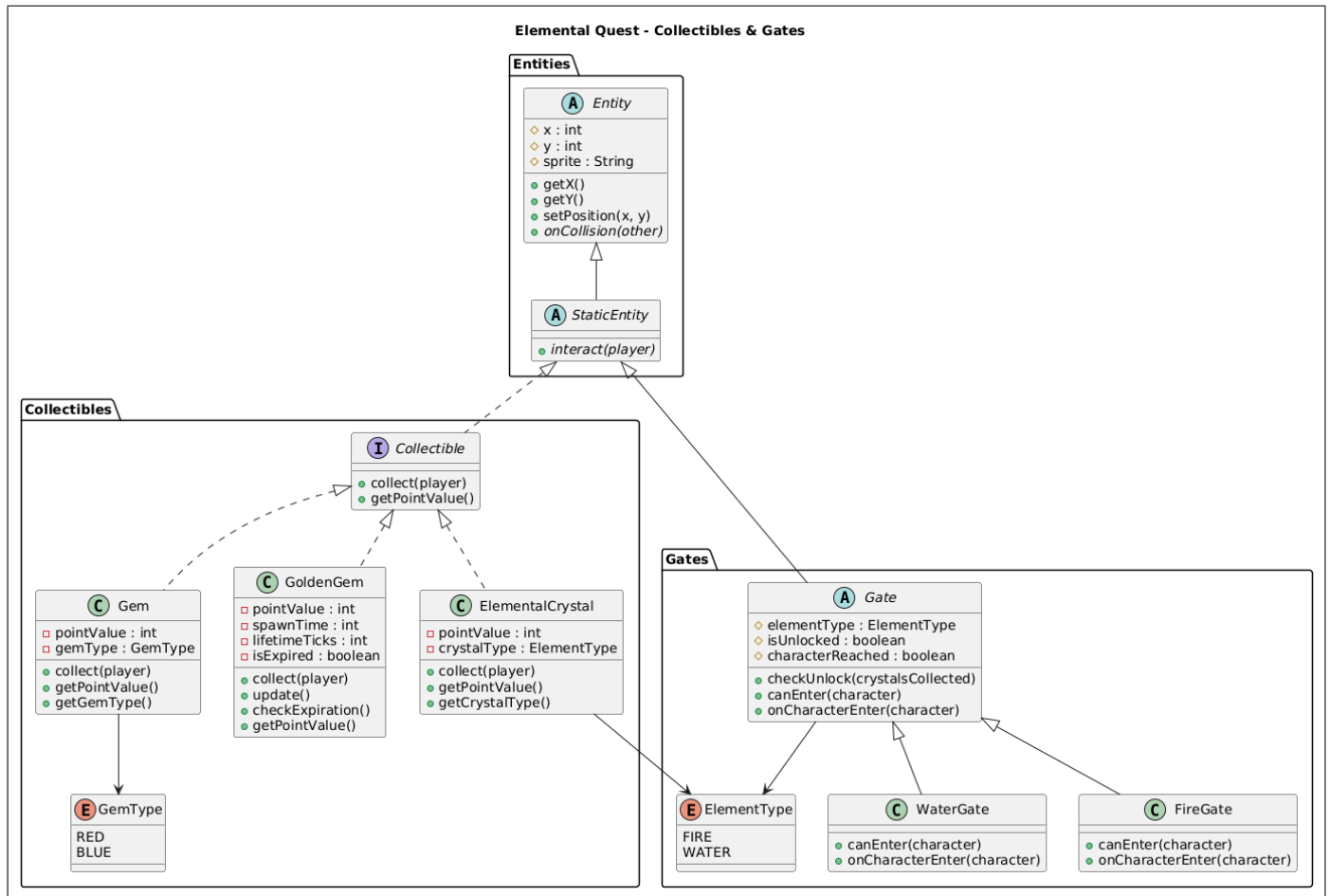


Figure 4: Collectibles and Gates showing element-based level completion mechanics

The collectibles and gates subsystems implement the core progression and completion mechanics through a clear interface-based design and element validation system. The Collectible interface defines a contract for all items that players can gather, ensuring uniform handling through the `collect` and `getPointValue` methods regardless of specific collectible type. ElementalCrystal represents required collectibles that must all be gathered to unlock the exit gates, with the Level class tracking collection progress against the `requiredCrystals` threshold. GoldenGem introduces time-based mechanics with `spawnTime`, `lifetimeTicks`, and `isExpired` attributes, demonstrating how temporary gameplay elements can be implemented through the `update` and `checkExpiration` methods. Stan-

dard Gems with GemType enumeration (RED, BLUE) provide variety and potential for future features like color-matching bonuses or element-specific scoring. The Gate abstract class enforces the dual-character victory condition by maintaining elementType, isUnlocked, and characterReached flags that validate both crystal collection and correct character arrival. FireGate accepts only Ember (FIRE element) while WaterGate accepts only Aqua (WATER element), implementing element-specific validation through the canEnter method that checks the character's ElementType against the gate's requirement. The Level's checkCompletion logic verifies that both gates have characterReached set to true and all crystals are collected before declaring victory, ensuring players must successfully navigate both characters through the level. This design extends naturally to additional elements by adding new Gate subclasses and ElementType values without modifying existing completion logic or collectible implementations.